

# A Reinforcement Learning Solution for Queue Management in Public Utility Services<sup>†</sup>

Todor Dobrev<sup>1</sup>, Miroslav Markov<sup>2</sup>  and Valentina Markova<sup>1,\*</sup> 

<sup>1</sup> Department of Communication Engineering and Technologies, Technical University of Varna, 9000 Varna, Bulgaria; tdobrev@tempus2000.com

<sup>2</sup> Department of Software and Internet Technologies, Technical University of Varna, 9000 Varna, Bulgaria; m.markov@tu-varna.bg

\* Correspondence: via@tu-varna.bg

<sup>†</sup> Presented at the International Conference on Electronics, Engineering Physics and Earth Science (EEPES 2025), Alexandroupolis, Greece, 18–20 June 2025.

## Abstract

This paper presents a reinforcement learning-based approach for optimizing queue management in public utility service environments. Using one year of real operational data from a utility office, a simulation model is developed to replicate daily service dynamics. A Q-learning agent is trained to allocate service types to counters dynamically, aiming to minimize client waiting time. The model treats the environment as a Markov Decision Process and uses an epsilon-greedy policy for learning optimal actions. Experimental results across multiple counter configurations demonstrate significant reductions in average waiting times, confirming the effectiveness and adaptability of the proposed method in dynamic service environments.

**Keywords:** queueing system; reinforcement learning algorithm; Q-learning; utility services

## 1. Introduction

The effective management of queues is a challenge for each entity dealing with large numbers of clients, patients, or visitors. The solutions vary widely, but most often they refer to queueing theory, rooted in the field of telecommunications and traffic engineering. It suggests that a model can be built so that waiting times and queue lengths could be predicted [1,2]. Given the vast diversity of fields as well as the specificities of each organization, the models, and their respective queueing strategies, formulating an adequate solution in each separate case could be different.

For services like emergency medical departments, overcrowding could have significant consequences not only on the patients' dissatisfaction but also on their health, and it can often be crucial for saving one's life. In such cases, issues like prioritizing cases and services is of primary importance [3]. On the other hand, in services like recreation and entertainment (e.g., theme parks) [4], the factors driving queue-related decisions could be based on clients' satisfaction (or the company's expected profit) and can sometimes allow waiting in the queue above some thresholds, compensated with a service of "great value" for the customer. However, the seasonal peaks of these services and group visits [5] could impose queue-optimization strategies based on a completely different logic.

In the context of public utility services, where the perceived added value of waiting is often limited, ensuring a smooth and time-efficient service experience becomes essential for maintaining user satisfaction. To achieve this, the selected queueing model in each specific



Academic Editor: Grigor Mihaylov

Published: 22 August 2025

**Citation:** Dobrev, T.; Markov, M.; Markova, V. A Reinforcement Learning Solution for Queue Management in Public Utility Services. *Eng. Proc.* **2025**, *104*, 6. <https://doi.org/10.3390/engproc2025104006>

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

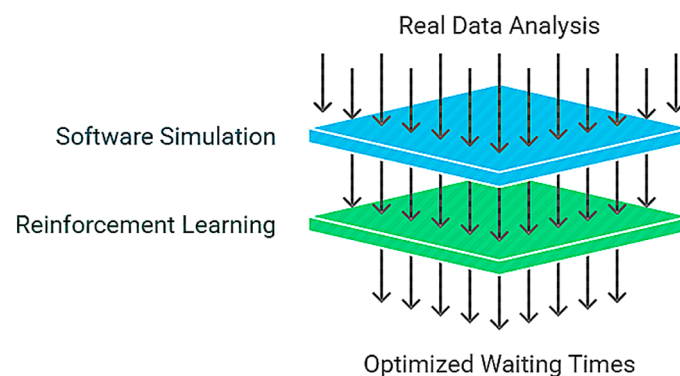
case should be informed by several key factors, outlined as follows: the estimated number of clients per day, the range of services provided by the entity, and the average processing time for each service; the number of counters/points of service (or servers, based on the queueing theory terminology) working in parallel; and the allocation of services or how they are distributed by counters and so on. Another issue is the number of queue stages, as well as the underlying principle of the service regarding the client stack.

Taking all these factors into account, in the current paper we propose a solution for the optimization of the average waiting time in a queueing system with several parallel working queues in a one-stage service scenario, which can be applied to a number of facility services. We base our solution on one-year real data recordings, a software simulation of the process, and the implementation of a reinforcement learning algorithm.

## 2. Method

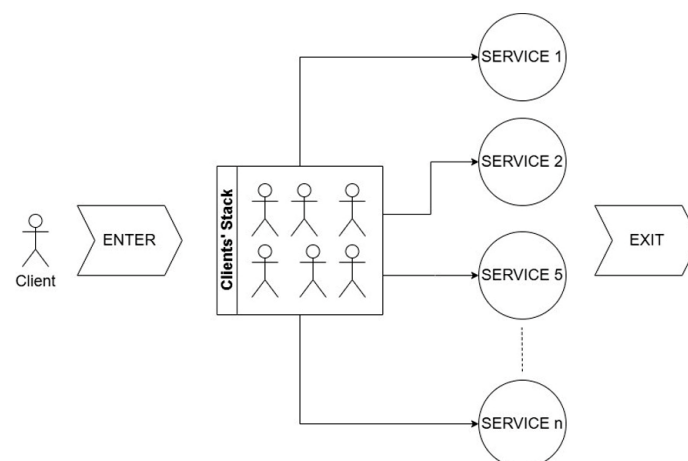
### 2.1. Method

The current paper bridges the gap between traditional queueing theory and reinforcement learning, introducing an innovative, real data-driven approach for optimizing service allocations in dynamic queue environments (Figure 1).



**Figure 1.** The approach for the optimization of queue waiting times in real public utility services.

The queueing system architecture (Figure 2) consists of a common client stack where the clients wait to be recalled for service, there is a known number of counters, and the allocation of services can vary in order to ensure the most effective process, optimizing the average waiting time. The processing of the stack generally follows the First In First Out (FIFO) rule, also known as First Come First Served (FCFS).



**Figure 2.** Queueing system architecture of the public utility service.

Unlike conventional queueing systems with fixed service assignments, this model leverages reinforcement learning to adaptively allocate counters based on real-time client arrivals and variable processing times. The allocation strategy is based on a learned service assignment policy, where each counter dynamically selects one service type to prioritize at each time step. All counters are equally capable of serving any request type, and the model uses Q-learning to optimize these assignments based on queue dynamics and system performance over time. The integration of real client arrival distributions and service time variability allows the Q-learning model to learn from empirical queue behavior, enhancing its real-world deployment potential.

## 2.2. Data

Data were gathered from an authentic source, more specifically, one of the offices of public utility Company X, over the course of a single calendar year. The dataset encompassed all clients served and all services rendered by the entity, including details such as arrival times, waiting durations, processing times, and the allocation of services and clients across operational counters.

## 2.3. Simulation

Subsequently, a simulation model was developed, incorporating input variables based on the real data extracted. The objective of the simulation was to replicate a process characteristic of a public utility service office. The model computes the average waiting time per client as this parameter is of paramount importance and is targeted for optimization.

## 2.4. Reinforcement Learning Model

In reinforcement learning models, an agent interacts with a specific environment in order to learn by their own experience and find the best decision-making policy. In our case we implemented a Q-learning algorithm (the agent), which interacted with the environment presented as a Markov Decision Process (MDP). The MDP could be described as follows:

- States  $S$ : Representing all possible queue configurations (how **service counters** are allocated to different service types);
- Actions  $A$ : Decisions regarding the allocation of services throughout counters;
- Transition function  $P(s'|s,a)$ : Probability of moving from state  $s$  to state  $s'$ , given action  $a$ .
- Reward function  $R(s,a)$ : Defines the immediate feedback for taking action  $a$  in state  $s$ .
- Discount factor  $\gamma$ : Balances future rewards vs. immediate rewards.

Since transitions depend on a probabilistic distribution (implemented in the script), the process follows the Markov property, outlined as follows: the next state only depends on the current state and action, not the past history. The reward function combines the following three components: a core efficiency term that penalizes deviation from the target queueing and waiting times, a stability penalty to discourage extreme operating conditions or counter misconfiguration, and a per-day penalty applied to any unserved clients, expressed as follows:

$$R(s,a) = R_{\text{efficiency}} + R_{\text{stability}} - P_{\text{unserved}} \quad (1)$$

This approach guides the agent toward configurations that are both effective and service complete.

The agent (Q-learning algorithm) observes the current state  $s$  and chooses an action  $a$  following the epsilon-greedy policy for balancing between exploration and exploitation. It explores a random action with probability  $\epsilon$  (initially 1, decaying over time) or exploits the best action known thus far from the Q-table with probability  $1 - \epsilon$ . After executing

an action, the agent receives a reward based on the queue's response. Then it updates the Q-value using the Bellman equation, expressed as follows:

$$Q(s,a) = (1 - \alpha)Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a')], \quad (2)$$

where the following hold:

$\alpha$ —learning rate (controls update speed);

$R$ —immediate reward;

$\gamma$ —discount factor;

$\max_{a'} Q(s',a')$ —best estimated future reward.

After updating the Q-value, the agent moves to the next state,  $s'$ . The process repeats until it meets the max iteration criteria.

### 3. Experimental Results

#### 3.1. Extracted Data

The data we used were collected during an interval of almost one calendar year from the middle of February 2024 to the end of January 2025 from one of the offices of Company X and presents the whole period since the ticketing system was adopted.

A summary of the extracted and analyzed data is presented in Table 1.

**Table 1.** Extracted data.

| Parameter                                       | Value                                      |
|-------------------------------------------------|--------------------------------------------|
| Average daily clients                           | 247                                        |
| Number of the services provided by the entity   | 7                                          |
| Distribution of clients across services         | [0.22, 0.19, 0.15, 0.20, 0.05, 0.11, 0.08] |
| Average processing time for service i (min.)    | [7, 7, 9, 10, 11, 12, 20]                  |
| Total number of counters operating concurrently | 12                                         |
| Mean interval time for client arrivals (min.)   | 2.3                                        |
| Standard deviation of interval time             | 0.03                                       |
| Average waiting time (min.)                     | 8.21                                       |
| Average total time in the system (min.)         | 19.16                                      |

#### 3.2. Implementation

The agent and the simulation environment were implemented in Matlab R2016a. The hyper-parameters of the model are depicted in Figure 1 along with their adjusted values.

The input parameters (Figure 3) were taken from the analysis of the extracted data, whereas the hyper-parameters of the Q-learning agent were experimentally adjusted. The number of iterations were chosen so as to represent the working hours (8 h or 480 min per day) over 20 days (roughly the number of working days in one calendar month). Each iteration represents a minute, after which the agent checks for free counters and suggests which service to allocate to them, implementing the FIFO principle for the clients chosen for the respective service.

#### 3.3. Results

Four scenarios with, respectively, 12, 10, 8, and 6 counters working in parallel are presented in Table 2.

The 12-counter scenario, compared to the extracted data with the same counter number performs better in terms of average waiting time and total time in the system, especially after the first 1000 iterations (roughly 2 days), when the behavior of the agent is shaped by the reward policy. The biggest decrease in percent is achieved with 10 counters (Table 2) and shows a steady behavior following initial training. Decreasing the number of counters

from this point onwards causes instability in the agent's behavior due to the insufficient operational capacity of the system and the respective unserved clients at the end of the working day.

```

%% Parameters
params.num_clients      = 247;           % Number of clients per day (8 hours) - Adjustable
params.total_minutes    = 480;           % 8-hour workday in minutes
params.max_iterations   = 9600;          % Maximum number of iterations (exactly 20 days, 20 * 480)
params.learning_rate    = 0.4;           % Learning rate for Q-learning
params.discount_factor   = 0.99;         % Discount factor
params.exploration_rate = 1.0;           % Initial exploration rate (increased for more exploration)
params.min_exploration  = 0.2;           % Increased minimum exploration rate for continued learning
params.exploration_decay = 0.9995;       % Slower decay for more exploration over time
params.moving_avg_window = 50;           % Moving average window for waiting time
params.avg_iterations    = 480 * 20;      % Last 20 days (1 to 9,600 iterations) for averages

% System parameters
num_counters            = 10;            % Number of counters (configurable, e.g., 9, 11, etc.)
service_times           = [7, 7, 9, 10, 11, 12, 20]; % Service times (minutes), allowing duplicates
probabilities            = [0.22, 0.19, 0.15, 0.20, 0.05, 0.11, 0.08]; % Service probabilities
num_services             = length(service_times);

params.mean_interarrival_time = 2.30;      % Mean interarrival time (minutes)
params.std_interarrival_time  = 0.03;      % Standard deviation of interarrival time

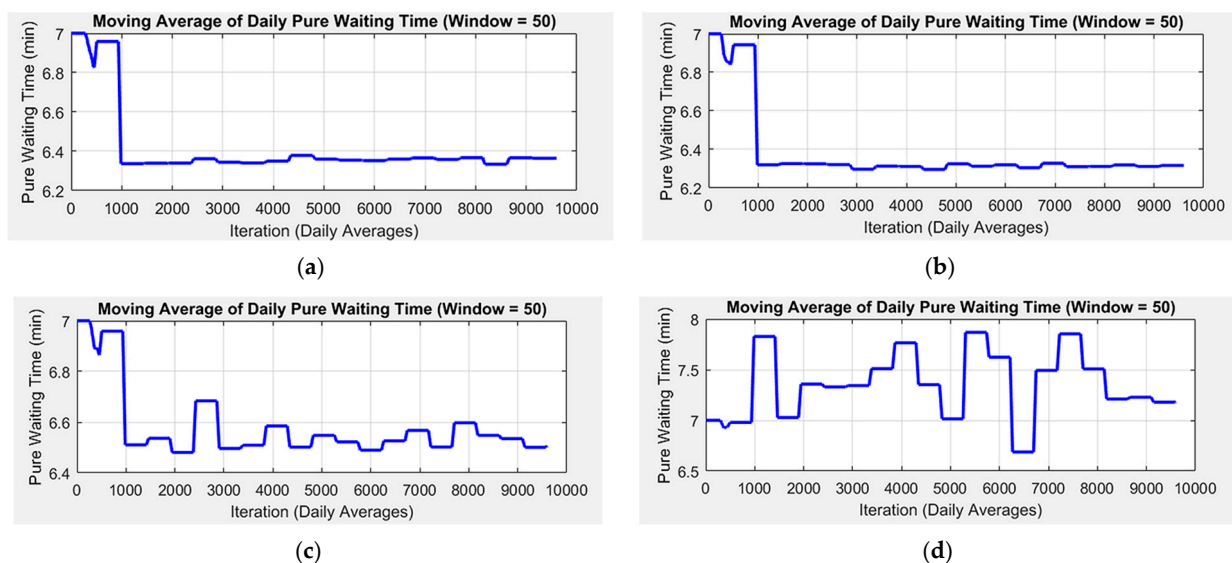
```

**Figure 3.** Input and system parameters of the Q-learning agent and its simulated environment.

**Table 2.** Waiting, service, and total times in the system in 4 scenarios on the first and last working days.

|                                        | 12 Counters | 10 Counters | 8 Counters | 6 Counters |
|----------------------------------------|-------------|-------------|------------|------------|
| Day 1 Avg Pure Waiting Time (min.)     | 6.96        | 6.94        | 6.96       | 6.98       |
| Day 1 Avg Service Time (min.)          | 9.63        | 9.36        | 9.04       | 9.52       |
| Day 1 Avg Total Time in System (min.)  | 16.59       | 16.30       | 16.00      | 16.50      |
| Day 20 Avg Pure Waiting Time (min.)    | 6.37        | 6.32        | 6.68       | 7.25       |
| Day 20 Avg Service Time (min.)         | 9.63        | 9.36        | 9.04       | 9.52       |
| Day 20 Avg Total Time in System (min.) | 16.00       | 15.67       | 15.72      | 16.77      |
| Avg Waiting Time Decrease (%)          | 8.50%       | 9.01%       | 4.04%      | −3.93%     |

The change in average waiting time across iterations is shown in Figure 4.



**Figure 4.** Training and behavior of the Q-learning agent during the 20-day period (9600 iterations): (a) 12 counters; (b) 10 counters; (c) 8 counters; and (d) 6 counters.

## 4. Conclusions

This study presents a data-driven reinforcement learning approach, leveraging Q-learning to optimize queue management in public utility services. By modeling the service environment as a Markov Decision Process and training the agent on real operational data, the system effectively reduces average client waiting times while dynamically adapting to varying workloads and resource constraints. The results demonstrate that the learned policies outperform baseline allocations across multiple counter configurations, with the most notable improvement observed in the 10-counter scenario. The Q-learning agent exhibits stable and efficient performance after a brief training phase, confirming the method's robustness and applicability. This approach contributes a scalable and adaptive solution for public service providers seeking to enhance efficiency and user satisfaction through intelligent queue management.

**Author Contributions:** Conceptualization, T.D., M.M. and V.M.; methodology, T.D., M.M. and V.M.; software, T.D. and M.M.; validation, T.D. and M.M.; formal analysis, T.D. and M.M.; investigation, T.D. and M.M.; resources, T.D.; data curation, T.D. and M.M.; writing—original draft preparation, T.D. and M.M.; writing—review and editing, V.M.; visualization, T.D.; supervision, V.M.; project administration, V.M.; funding acquisition, V.M. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Institutional Review Board Statement:** Not applicable.

**Informed Consent Statement:** Not applicable.

**Data Availability Statement:** The dataset can be made available upon request from the authors.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. Koko, M.A.; Burodo, M.S.; Suleiman, S. Queuing Theory and Its Application Analysis on BusServices Using Single Server and Multiple Servers Model. *Am. J. Oper. Manag. Inf. Syst.* **2018**, *3*, 81–85. [[CrossRef](#)]
2. Johri, P.; Misra, A. How Queuing Theory Does Correlate with Our Lives-An Analysis. *J. Basic Appl. Eng. Res.* **2016**, *3*, 870–872.
3. Cildoz, M.; Ibarra, A.; Mallor, F. Accumulating Priority Queues versus Pure Priority Queues for Managing Patients in Emergency Departments. *Oper. Res. Health Care* **2019**, *23*, 100224. [[CrossRef](#)]
4. Li, J.; Li, Q. Analysis of Queue Management in Theme Parks Introducing the Fast Pass System. *Heliyon* **2023**, *9*, e18001. [[CrossRef](#)] [[PubMed](#)]
5. Hsieh, Y.-C.; You, P.-S. A Comparison of Three Evolutionary Algorithms for Group Scheduling in Theme Parks with Multitype Facilities. *Sci. Prog.* **2024**, *107*, 00368504241278424. [[CrossRef](#)] [[PubMed](#)]

**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.